

Rapport final

PFR2

2026

Université de Toulouse – Faculté des sciences de l'ingénierie

Groupe 5 : Antonin Gerain, Lucas Haegel, Adrien Nosel, Ny Aina Raharitsifa.

Client : Isabelle Ferrané



Sommaire

Introduction.....	3
I. Architectures du projet.....	4
I.1 – Architecture du robot.....	4
I.2 – Architecture des modules.....	5
II. Fonctionnalités développées.....	6
II.1 – Pilotage du robot.....	6
II.2 – Application Bluetooth.....	10
II.3 – LiDAR.....	11
II.4 – Traitement de l'image.....	12
II.5 – Interface web.....	20
III. Bilan individuel.....	22

Introduction

Ce projet a pour objectif de développer une plateforme mobile pouvant se déplacer de façon autonome, semi-autonome ou manuelle. Durant la phase du PFR1, nous avons développé des modules logiciels permettant de simuler le déplacement du robot avec différentes fonctionnalités comme le traitement de requêtes textuelles et vocales, la création d'un espace virtuel, le traitement d'images et la création d'une interface.

Pour ce qui est du PFR2, l'objectif est de repartir des fonctionnalités développées lors du PFR1 pour les améliorer ou les modifier puis les implémenter dans une plateforme mobile physique et ainsi se concentrer sur l'aspect de l'informatique embarquée et sur l'utilisation de différents composants mis à disposition.

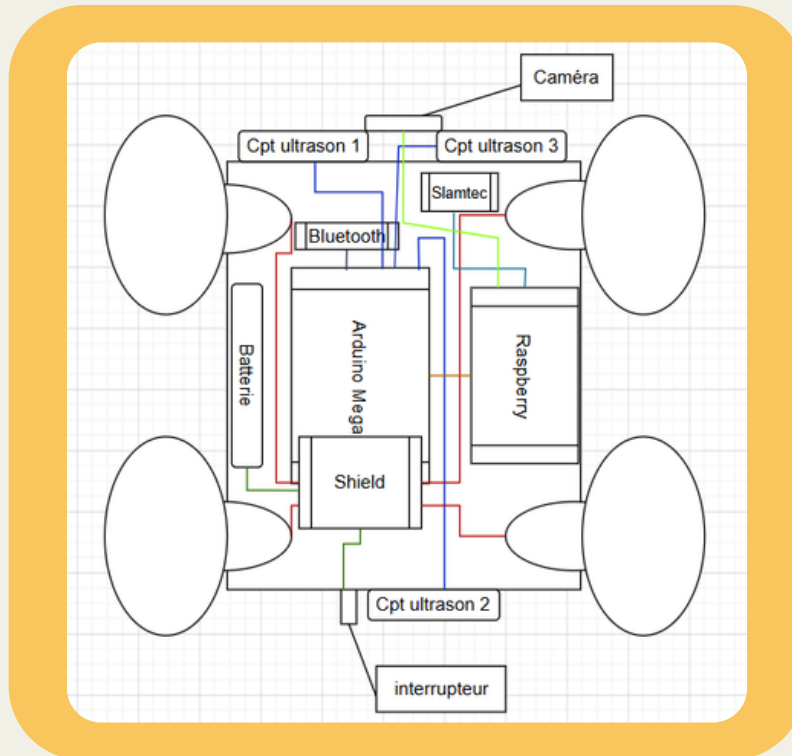
L'organisation du PFR2 est différente de celle mise en place durant le PFR1 car le travail se fait durant des créneaux précis dans l'emploi du temps avec un suivi régulier auprès du client grâce aux réunions chaque matin et chaque soir. Cette méthode de travail nous a permis d'établir des objectifs précis chaque jour et ainsi gagner en efficacité.

Le PFR2 est un défi significatif pour notre groupe car nous avons dû nous réorganiser sur la répartition des tâches en raison d'une modification de la composition du groupe. Aussi, même si certaines bases du PFR1 ont pu être réutilisées, nous avons dû repenser certaines fonctionnalités pour respecter le cahier des charges final. Une grande partie de ce projet a également été concentrée sur la recherche liée à l'utilisation des composants, étant donné que la plupart d'entre nous n'en avait jamais utilisé, ainsi que sur le développement informatique pour l'intégration des équipements dans le code.

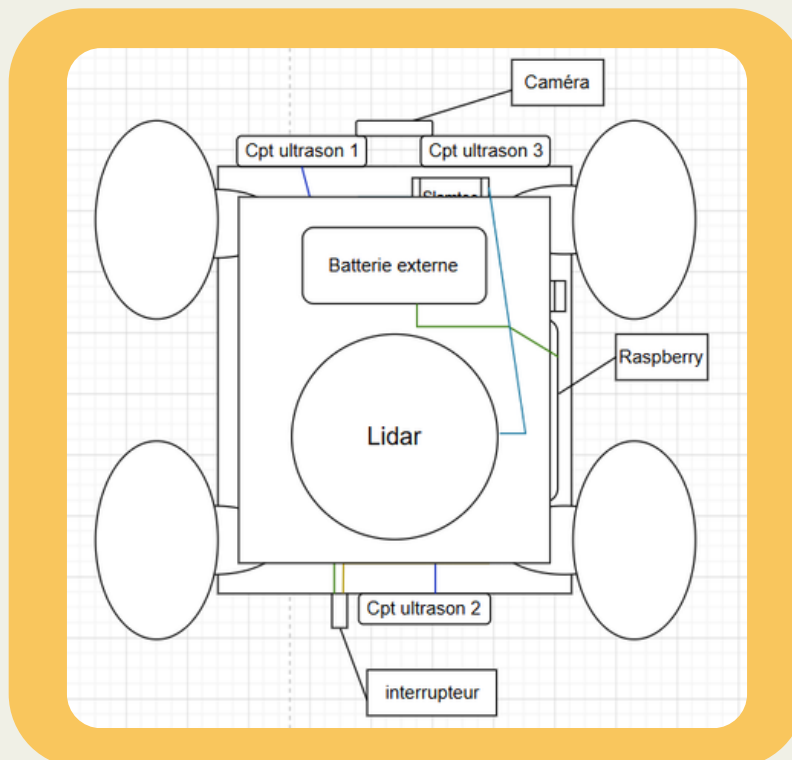
Ce rapport est divisé en trois parties distinctes : la première partie concerne l'architecture du robot, la deuxième partie détaille chaque fonctionnalité que l'on a développée et la troisième partie évoque le bilan personnel de chaque membre du groupe. Pour clôturer le rapport, la conclusion porte sur les améliorations possibles du projet si celui-ci était repris par d'autres étudiants.

I. Architectures du projet

I.1. Architecture du robot



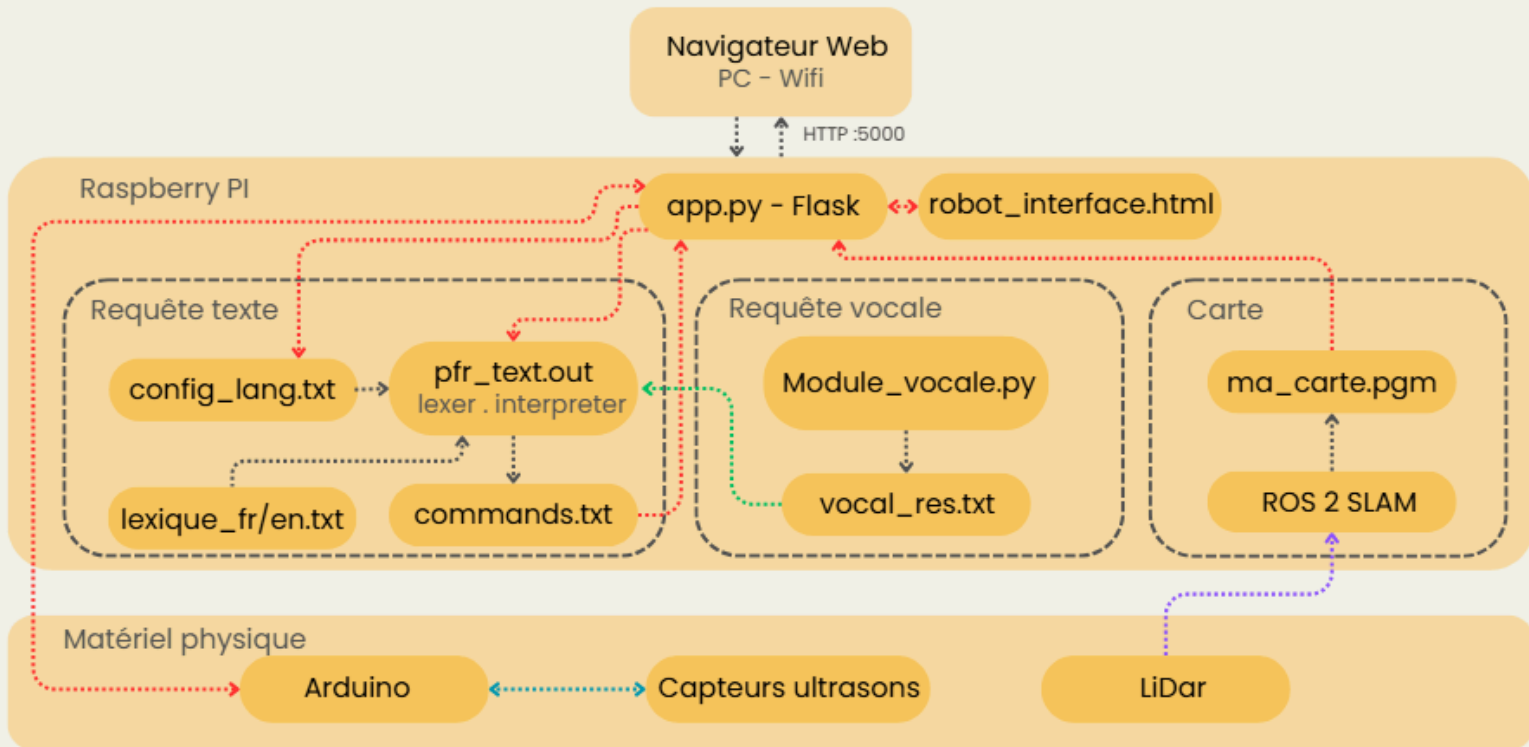
Architecture interne du robot



Architecture externe du robot

I. Architectures du projet

I.2. Architecture des modules



Architecture schématisée des modules

Tous les modules ont été intégrés de manière à ce que le serveur Flask (`app.py`) soit au centre du projet. Celui-ci est la passerelle permettant à tous les composants et programmes de communiquer entre eux et ainsi permettre à l'utilisateur de pouvoir utiliser ces modules. Nous avons fait le choix de rendre la Raspberry Pi le cerveau du robot permettant ainsi d'avoir une plateforme utilisable depuis n'importe quel appareil connecté à son réseau.

L'architecture se décompose en trois niveaux distincts. Le premier niveau est celui de l'interface utilisateur, accessible depuis n'importe quel navigateur web via une connexion HTTP sur le port 5000. Celui-ci communique directement avec le serveur Flask à travers la page `robot_interface.html`.

Le deuxième niveau est celui des modules logiciels tournant sur la Raspberry Pi. On y retrouve trois fonctionnalités principales : le module de requête texte, le module de requête vocale et le module de cartographie.

Enfin, le troisième niveau est celui du matériel physique, composé d'un Arduino, de capteurs ultrasons et d'un LiDAR, qui interagissent avec les modules logiciels pour les déplacements et la perception de l'environnement.

II. Fonctionnalités développées

II.1. Pilotage du robot

Le pilotage du robot est une fonctionnalité qui permet à l'utilisateur de contrôler les déplacements de la plateforme mobile selon trois modes distincts

: le pilotage manuel, le pilotage par requête textuelle sur clavier et le pilotage par commande vocale. Ces trois modes ont été développés lors du PFR1 sous forme simulée, puis adaptés et intégrés dans le robot physique durant le PFR2. Les commandes transitent par un serveur Flask tournant sur la Raspberry Pi, puis sont transmises à l'Arduino via une liaison série USB.

Modes de pilotage

Pilotage manuel

Le pilotage manuel permet à l'utilisateur de contrôler le robot en temps réel depuis l'interface web, via un D-Pad virtuel ou les touches du clavier. Les touches Z, Q, S, D (ou les flèches directionnelles) correspondent respectivement aux directions avant, gauche, arrière et droite. Un bouton d'arrêt d'urgence est également disponible, accessible via la barre espace ou la touche Échap.

Ce mode fonctionne en maintien : le robot avance tant que la touche est enfoncée et s'arrête dès qu'elle est relâchée. Une sécurité supplémentaire a été intégrée pour envoyer automatiquement un ordre d'arrêt si l'utilisateur perd le focus de la page, évitant ainsi que le robot continue de se déplacer sans contrôle. Un second panneau de commande précise permet quant à lui d'envoyer une valeur exacte de distance en mètres ou d'angle en degrés, offrant un contrôle plus fin des déplacements.

Chaque action de l'utilisateur génère une requête HTTP vers Flask, qui construit la commande au format texte et la transmet à l'Arduino via le port série, selon le protocole suivant : forward 2.00 pour avancer de 2 mètres, left 90.00 pour tourner à gauche de 90 degrés, ou stop 0 pour un arrêt immédiat.

II.1. Pilotage du robot

Pilotage par requête textuelle

Ce mode permet à l'utilisateur d'envoyer des instructions en langage naturel, en français ou en anglais, directement depuis l'interface web. Par exemple : "avance de 2 mètres puis tourne à droite".

Flask transmet la phrase au programme `pfr_text.out`, un exécutable compilé en C qui effectue deux opérations successives. La première est une analyse lexicale : chaque mot de la phrase est comparé au lexique correspondant à la langue choisie et classé par type — action, direction, nombre, unité, lien grammatical, etc. La seconde est une interprétation : les mots classés sont assemblés en commandes concrètes. Ainsi, "avance" associé à "2" et "mètres" produit la commande `forward 2.00`. Les commandes générées sont écrites dans un fichier intermédiaire `commands.txt`, que Flask lit ensuite pour les envoyer séquentiellement à l'Arduino.

Certains mots-clés déclenchent des séquences prédéfinies plus complexes, comme le demi-tour (`left 180.00`), le zigzag (séquence de quatre mouvements alternés) ou encore le cercle (huit répétitions d'un avancer suivi d'une rotation de 45 degrés). Le choix de la langue est géré via un fichier `config_lang.txt`, que Flask met à jour avant chaque appel au moteur C, et que ce dernier lit au démarrage pour charger le bon lexique.

Pilotage par commande vocale

Le pilotage vocal a connu deux versions successives au cours du projet, reflétant l'évolution de l'architecture entre la phase de développement et la phase d'intégration finale.

Dans un premier temps, la reconnaissance vocale était assurée par un script Python dédié, `Module_vocal.py`, tournant directement sur la Raspberry Pi. Ce script utilisait la bibliothèque `SpeechRecognition` pour écouter le microphone branché sur la Raspberry Pi, transmettre le flux audio aux serveurs de Google et récupérer la transcription textuelle. Une fois la phrase reconnue, elle était écrite dans un fichier intermédiaire `vocal_res.txt`, que Flask lisait ensuite pour lancer le traitement via le moteur C. Cette approche a été utile durant la phase de développement et de test des modules de manière indépendante, avant que l'interface web ne soit finalisée.

II.1. Pilotage du robot

Elle présentait cependant des contraintes pratiques lors de l'intégration : elle nécessitait un microphone physiquement branché sur la Raspberry Pi, et le script devait être lancé en sous-processus par Flask, ce qui rendait la gestion des erreurs et des timeouts plus complexe.

C'est pourquoi, lors de l'intégration finale dans l'interface web, la reconnaissance vocale a été déplacée côté navigateur, en s'appuyant sur l'API Web Speech, disponible nativement sur Chrome et Edge. Lorsque l'utilisateur clique sur le bouton microphone dans l'interface, le navigateur active directement le micro de l'appareil utilisé, envoie le flux audio à Google et retourne la transcription textuelle sans aucun traitement supplémentaire sur la Raspberry Pi. Cette solution s'est révélée plus simple à intégrer et surtout plus pratique à utiliser, puisqu'un simple smartphone ou ordinateur peut servir de microphone.

Exécution sur l'Arduino et calibration

L'Arduino reçoit les commandes sur son port série, les décode et les exécute. Ne disposant pas d'encodeurs de roues, il ne peut pas mesurer directement la distance parcourue ou l'angle de rotation. Il traduit donc chaque commande en une durée de fonctionnement des moteurs, en s'appuyant sur deux constantes de calibration déterminées expérimentalement :

- `DISTANCE_RATIO` = 0.0007 : le robot parcourt 0,0007 mètre par milliseconde, soit environ 0,7 m/s.
- `ANGLE_RATIO` = 0.038 : le robot tourne de 0,038 degré par milliseconde, soit environ 38 degrés par seconde.

La durée de fonctionnement des moteurs est calculée par la formule $\text{durée (ms)} = \text{valeur demandée} / \text{ratio}$. Par exemple, pour avancer de 2 mètres : $2,0 / 0,0007 = 2857$ ms, soit environ 2,9 secondes de fonctionnement des moteurs. Ces constantes ont été obtenues par une série de tests pratiques : le robot a été lancé sur une distance et un angle connus, puis les valeurs ont été ajustées progressivement jusqu'à obtenir un comportement satisfaisant.

Cette approche suppose une vitesse constante des moteurs, ce qui constitue sa principale limite. En pratique, le niveau de charge de la batterie, la nature du sol ou le poids embarqué peuvent faire dériver la trajectoire réelle par rapport à la trajectoire théorique. Une recalibration des constantes est donc nécessaire si les conditions d'utilisation changent significativement.

II.1. Pilotage du robot

Gestion des obstacles

Pour assurer la sécurité des déplacements, trois capteurs ultrason ont été intégrés : deux à l'avant (gauche et droite) et un à l'arrière. Ils mesurent en continu la distance aux obstacles environnants et transmettent leurs relevés à la Raspberry Pi toutes les 500 millisecondes via le port série. Ces capteurs jouent un rôle actif dans le comportement du robot grâce à une machine à états implémentée directement dans le code Arduino. Ce mécanisme permet au robot de réagir aux obstacles de manière autonome, sans intervention de l'utilisateur, et sans jamais bloquer l'exécution du reste du programme.

Le comportement varie selon la direction du déplacement. En marche arrière, si le capteur arrière détecte un obstacle à moins de 30 centimètres, le robot s'arrête immédiatement. En marche avant, la réaction est plus élaborée : une séquence d'évitement automatique en quatre phases est déclenchée. La première phase consiste à reculer brièvement pendant 300 millisecondes pour dégager l'espace devant le robot. Une courte pause de 150 millisecondes suit afin de stabiliser le mouvement. Le robot effectue ensuite une rotation pendant 750 millisecondes, dont la direction est choisie intelligemment : si le capteur avant gauche voit plus loin que l'avant droit, le robot tourne à droite, et inversement. Enfin, avant de reprendre l'avance, le robot vérifie que la voie est effectivement libre. Si un obstacle est toujours détecté, la séquence est relancée dans la direction opposée.

Cette gestion autonome des obstacles permet au robot de continuer à progresser même en cas d'imprévus dans son environnement, ce qui est particulièrement utile lors des déplacements déclenchés par les modes textuel et vocal, où l'utilisateur n'a pas de visibilité directe sur le terrain.

II.2. Application Bluetooth

Branchement du composant HM10

Le composant HM10 est connecté à la carte MEGA 2560 avec la broche TX du composant bluetooth sur la broche RX de la MEGA 2560 et la broche RX du composant bluetooth sur la broche TX de la carte MEGA 2560. Ensuite la broche d'alimentation et de terre sont respectivement branchées sur la broche S+ et S- du Shield L293D.

Connexion en BLE (Bluetooth Low Energy)

Pour pouvoir piloter le robot à distance par bluetooth, la plateforme mobile est équipée du composant HM10 qui permet de transmettre et de recevoir des informations en utilisant la technologie Bluetooth Low Energy, ce qui est arrangeant car le BLE consomme très peu d'énergie car il échange des informations que quand il en a besoin contrairement à du bluetooth classique qui envoie en permanence des informations. La plateforme mobile ayant des ressources limitées, cette technologie est adaptée.

L'utilisation du BLE se gère à l'aide d'applications prévues pour gérer cette technologie. C'est donc Serial Bluetooth Terminal, une application mobile, qui est utilisée pour communiquer avec le composant HM10.

Déplacements de la plateforme mobile

Une fois sur l'application Serial Bluetooth Terminal, une configuration est nécessaire. Tout d'abord, dans l'onglet "Settings" puis "Send" il faut mettre "Newline" en None. Cela va désactiver l'envoi de caractères de retour à la ligne qui parasitent la gestion des informations par la carte MEGA 2560.

Ensuite il suffit de connecter le téléphone à la plateforme mobile grâce à l'onglet "Devices" puis "Bluetooth LE".

Les déplacements sont ensuite gérés dans l'onglet "Terminal" où des boutons sont paramétrés pour envoyer des valeurs comprises entre 1 et 5 en hexadécimal pour faire respectivement avancer, reculer, aller à gauche, aller à droite et stopper la plateforme.

II.3. LiDAR

Fonctionnement et utilisation du lidar

Pour cartographier l'environnement, la plateforme mobile est équipée d'un RPLidar A1 qui en envoyant des lasers tout en tournant va pouvoir nous retourner la position de tous les éléments touchés par les lasers dans un espace 2D. Une image de l'environnement est créée et affichée sur l'interface web.

Utilisation de SLAM

Pour pouvoir utiliser les données reçues par le lidar, plusieurs solutions étaient possible à créer de A à Z les programmes pour construire une map de l'environnement mais cela aurait été trop complexe et long à mettre en place, ou utiliser des technologies comme RTAB-MAP qui possède déjà plein de librairies et qui sont donc plus simples et rapides à mettre en place mais qui consomment beaucoup de ressources.

Le SLAM a semblé être le plus adapté pour fonctionner avec la plateforme mobile, il est peu coûteux en ressources parmi les autres technologies disponibles, est bien documenté et souvent mis à jour ce qui rend plus facile son intégration et s'exécute assez rapidement sur la raspberry pi 3B+.

Pour construire une map, le SLAM va donc faire un scan de l'environnement et récupérer un nuage de points avec les distances de tous les objets puis utilise une grille avec la probabilité des points de l'environnement de la présence ou non d'un objet (0.0 pour libre, 0.5 pour inconnu, 1.0 pour obstacle). Ensuite SLAM effectue une transformation pour estimer le déplacement du robot dans l'environnement puis corrige les formes déjà connues s'il y a un changement.

Mise en place de l'environnement

Pour utiliser le SLAM, il faut configurer la raspberry. Il faut mettre en place ROS2 qui va implémenter le driver USB pour le lidar et les librairies nécessaires aux scans et calculs. Celui-ci tournant sous Ubuntu et la raspberry étant sur une Debian, un docker est mis en place pour accueillir ROS2. Ensuite dans ce même docker, les librairies de python et de slam_toolbox sont installées.

Il suffit ensuite d'utiliser un programme en python pour lancer l'analyse de l'environnement et de capturer l'image.

II.4. Traitement de l'image

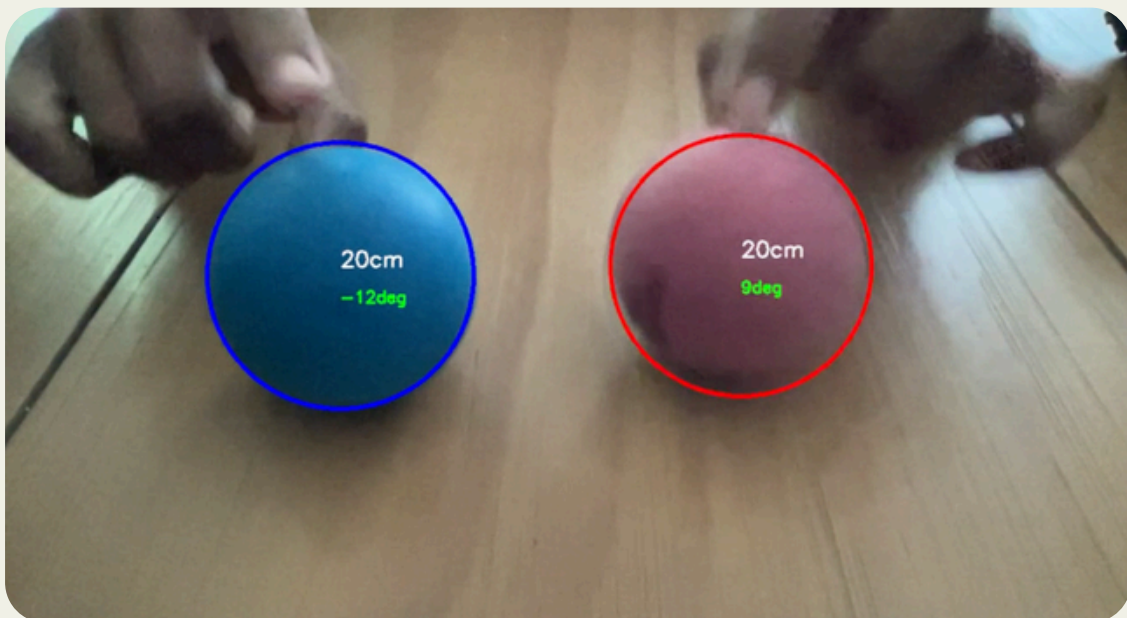
Introduction et Objectifs

Dans le cadre du PFR2, le module de traitement d'image a pour objectif d'agir comme le "système visuel" du robot. La mission principale de ce sous-système est double :

Détecter et identifier une balle spécifique dans l'environnement en fonction de sa forme (pour ne détecter que des balles) et de sa couleur (pour détecter la bonne balle).

Localiser cette cible en temps réel pour fournir au robot des instructions de navigation claires (distance à parcourir en centimètres et angle de rotation en degrés).

La partie logicielle de ce module est aujourd'hui une réussite complète. L'algorithme final (V8) remplit l'intégralité du cahier des charges de manière robuste et en temps réel. Ce rapport retrace l'évolution itérative du code.



II.4. Traitement de l'image

Évolution de l'Algorithme : De la détection basique à la robustesse

Le développement a débuté par des tests sur les images fournies au PFR1 afin de valider les concepts de base du traitement colorimétrique à l'aide de la bibliothèque OpenCV.

image_VI.py

L'objectif était l'apprentissage de la manipulation de données par OpenCV. Dans un premier temps, charger une image et en extraire les informations basiques (dimensions, canaux) pour ensuite envisager la conversion de l'espace colorimétrique RGB vers HSV, dans le but d'appliquer un masque pour isoler les pixels correspondant à des plages de couleurs théoriques (trouvées sur internet). Un histogramme simplifié permettait de compter ces pixels pour émettre une hypothèse sur la présence d'une balle d'une certaine couleur. Cependant, cette version ne vérifiait pas la géométrie de l'objet détecté.

```
Image chargée avec succès !
Dimensions : 300x300 pixels
Nombre de canaux (couleurs) : 3
--- Histogramme des couleurs ---
Pixels Jaunes : 3005
Pixels Bleus : 0
Pixels Rouges : 220
ALERTE : Balle jaune suspectée !
Appuie sur une touche.
```

Qu'est-ce que le HSV et pourquoi le choisir ?

Le format HSV sépare la couleur pure (Hue) de son intensité (Saturation) et de sa luminosité (Value). On le choisit à la place du RGB car il est insensible aux changements de lumière. Même si une ombre passe sur la balle, sa teinte (H) reste la même, ce qui rend la détection infiniment plus robuste.

Comment OpenCV passe de RGB à HSV ?

Via la fonction `cv2.cvtColor()`, OpenCV applique des formules mathématiques pixel par pixel :

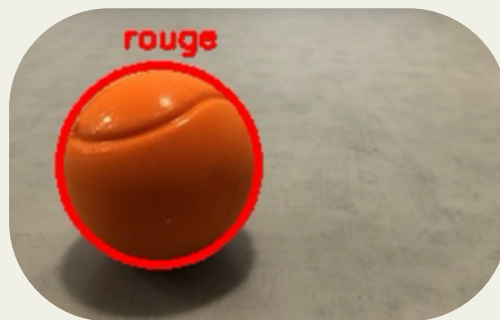
- Pour V (Valeur) : Il prend la valeur maximale parmi le Rouge, Vert et Bleu.
- Pour S (Saturation) : Il calcule l'écart entre le max et le min du RGB.
- Pour H (Teinte) : Il calcule l'angle de la couleur sur un cercle chromatique (ramené sur une plage de 0 à 179 pour coder l'information sur 8 bits).

II.4. Traitement de l'image

image_V2.py

Le programme utilise la fonction `cv2.findContours` pour identifier les objets colorés et trace un cercle circonscrit autour de l'objet détecté pour valider visuellement la détection.

À cette étape du développement, le programme se trompe sur environ 10% des images données. Le programme manque de robustesse dans la détection des couleurs due au choix des valeurs HSV et dans la détection des formes due à la forme de la balle elle-même. Comme l'on peut le voir ci-dessous, la balle n'est pas uniformément "rouge" (il y a un dégradé), de plus le design "balle de tennis" fait que la balle est traversée d'un liseré foncé (car dans l'ombre).



image_V3.py

Automatisation du traitement par lots pour analyser un grand nombre d'images, y compris celles présentant des conditions d'éclairage complexes (ombres, reflets). C'est ici que les filtres morphologiques (ouverture et fermeture) ont été implémentés pour "nettoyer" le masque binaire, boucher les trous causés par les ombres et éliminer le bruit de fond, permettant ainsi de reconstituer une balle "propre" numériquement.

Avec ces modifications, le programme trouve les balles de la bonne couleur sur 100% des images fournies pour le projet. Dans la suite, nous souhaiterions être capables de détecter grâce à des images prises par une caméra, voire dans d'autres versions grâce à un flux vidéo.



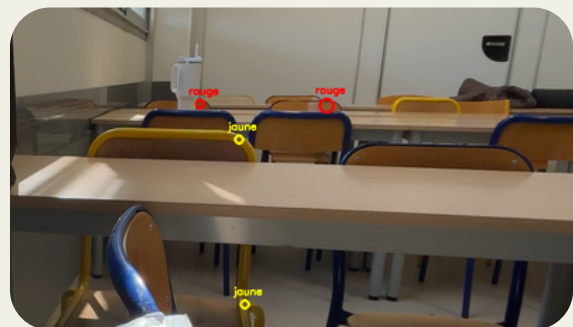
II.4. Traitement de l'image

Temps réel et Calibration

Le passage au flux vidéo en direct (webcam) a soulevé de nouveaux défis liés à la fidélité de la détection.

image_V4.py

Implémentation du flux vidéo en direct. Si le temps réel fonctionnait, l'algorithme générerait des faux positifs en détectant tout objet s'approchant de la couleur ciblée, peu importe sa forme. Deux conclusions se sont imposées : les valeurs HSV génériques n'étaient pas adaptées à l'éclairage réel, et la validation de la forme devait être mathématiquement durcie.



reglage_hsv.py

L'outil de calibration (reglage_hsv.py) : Pour résoudre le problème colorimétrique, un programme annexe a été développé. Équipé d'une interface graphique avec des curseurs (trackbars), il a permis d'ajuster manuellement les seuils (Teinte, Saturation, Valeur) en temps réel sur une image de référence. L'objectif était d'obtenir un masque parfait (la balle en blanc pur sur un fond totalement noir).

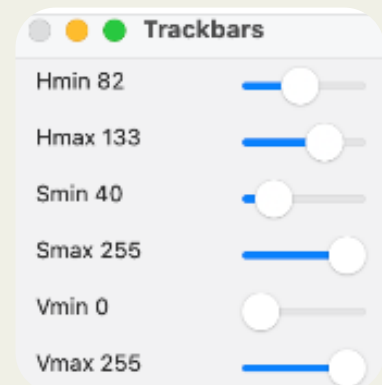
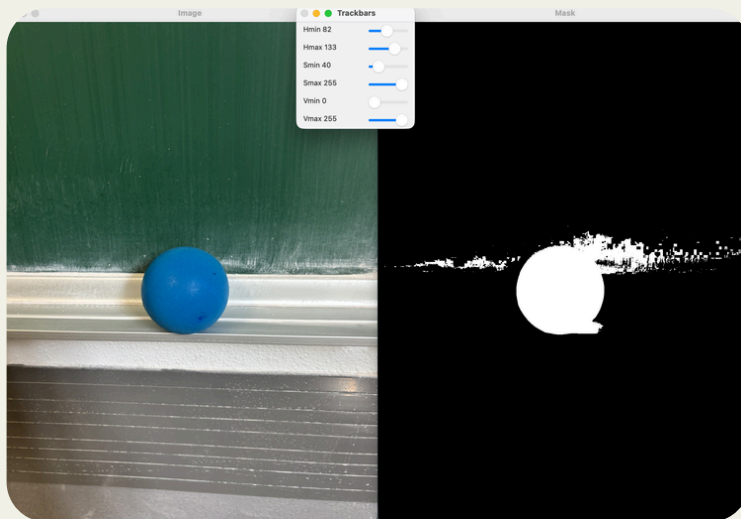
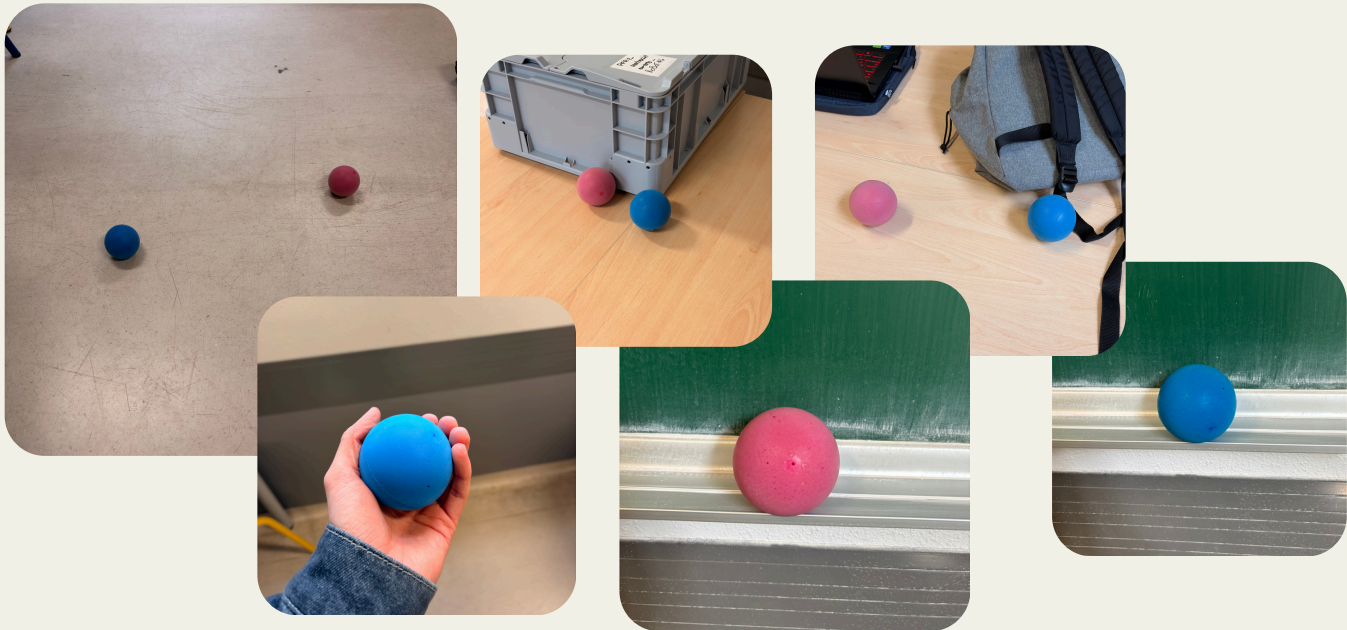
Après avoir fait des recherches sur le sujet, nous sommes arrivés au protocole suivant : $V \rightarrow S \rightarrow H$

- **V (Luminosité)** : On commence par ouvrir la plage au maximum (0-255) pour voir l'objet, puis on réduit le Vmin pour supprimer le noir/fond sombre sans perdre la balle.
- **S (Saturation)** : On réduit le Smin pour éliminer les zones trop grises ou blanches du décor, tout en gardant la couleur vive de la balle.
- **H (Teinte)** : Une fois que l'objet est bien isolé, on resserre la plage Hmin-Hmax sur la couleur précise (ex: 100-120 pour le bleu). C'est le réglage final qui définit la couleur cible.

Pourquoi cet ordre et pas un autre ? Régler la Teinte (H) en premier est inutile si la luminosité (V) ou la pureté (S) du signal est polluée par le bruit de l'image. On nettoie d'abord l'intensité lumineuse avant de choisir la couleur.

Une fois les mesures effectuées sur des balles dans toutes sortes de situations, nous avons éliminé les cas trop extrêmes et défini les valeurs min (minimin parmi les tests) et max (maximax parmi les tests) comme nos valeurs seuil.

II.4. Traitement de l'image



Rouge						Bleu					
H		S		V		H		S		V	
Bas	Haut	Bas	Haut	Bas	Haut	Bas	Haut	Bas	Haut	Bas	Haut
168	179	63	255	0	255	90	111	101	255	0	255
114	179	35	255	143	255	100	109	136	255	54	255
165	179	85	255	38	255	97	118	137	255	77	255
171	179	118	255	0	255	101	114	174	255	47	255

II.4. Traitement de l'image

image_V5.py

image_V6.py

L'intégration des nouvelles plages HSV calibrées et d'un double filtre mathématique sur la forme (calcul de la circularité parfaite et du ratio d'occupation de l'aire) a permis d'obtenir une détection exclusive : le programme ne détecte désormais que les balles, rejetant les autres objets de même couleur.

Localisation spatiale

Une fois la détection visuelle fiabilisée, l'enjeu était de traduire la position de la balle sur l'image en coordonnées réelles pour guider le robot.

image_V7.py

En utilisant le théorème de Thalès et les caractéristiques optiques de la caméra (focale, taille du capteur), un calcul de grandissement a été implémenté pour estimer la distance de l'objet.

A ce stade du projet, la caméra n'était plus détectée par la Raspberry. Nous n'avons ni trouvé la raison de ce dysfonctionnement, ni trouvé de solution. Nous avons alors développé notre logiciel de traitement de l'image à l'aide de la caméra smartphone et non de celle embarquée sur le robot. Toutes les spécifications (focale, taille du capteur, etc.) sont alors celles du smartphone et devront être adaptées dans le code final. D'où l'importance de ne rien rentrer en dur dans le code. Afin de répondre à cette exigence, nous avons eu l'idée de développer une librairie 'smartphone' et une autre 'arducam' afin d'utiliser des variables qui, dépendant de la librairie importée, allaient utiliser l'une ou l'autre des caméras. Cependant, le poids du développement de ces solutions pour le peu de paramètres à changer pour passer d'une caméra à l'autre nous a fait abandonner cette approche.

Un problème a été identifié : le cercle englobant dessiné par OpenCV prenait un "halo" autour de la balle, la faisant paraître plus grosse en pixels, ce qui faussait le calcul et la faisait apparaître plus proche qu'elle ne l'était vraiment.



II.4. Traitement de l'image

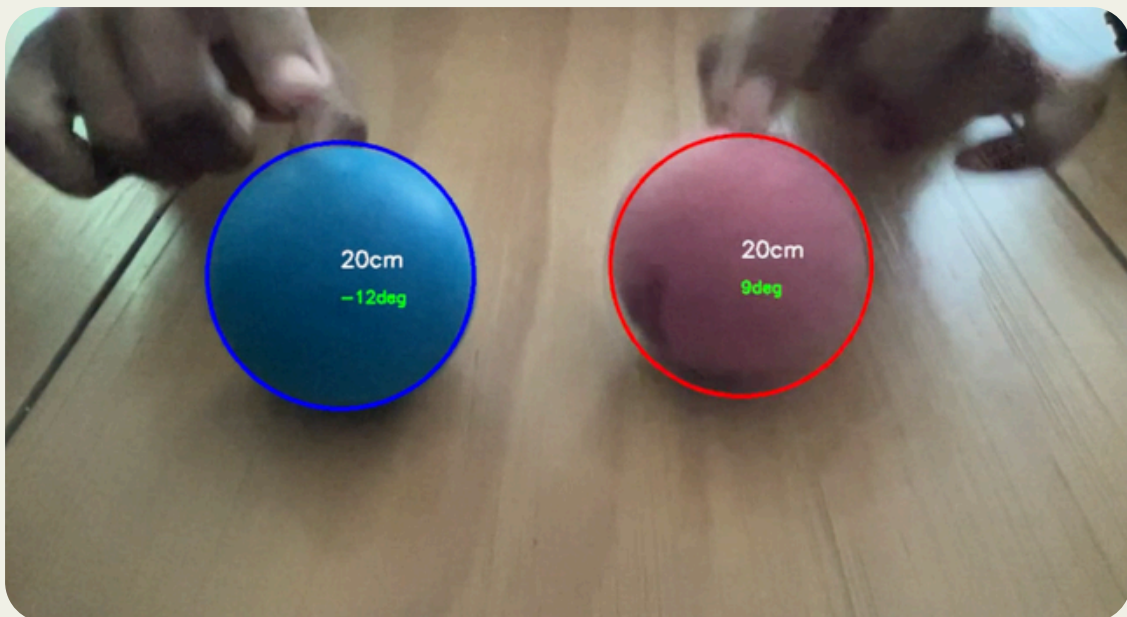
image_V8.py

Cette version est l'aboutissement logiciel du projet.

Correction de distance : Une étape d'érosion mathématique (CORRECTION_HALO) a été ajoutée pour annuler l'effet de halo. Suite à cela, l'erreur apparaissait plus identifiable et a donc été identifiée comme linéaire de pente 1,5. Nous avons donc appliqué un gain proportionnel linéaire ($K = 1,5$) pour corriger l'erreur résiduelle de la focale. La distance mesurée est désormais très précise et cohérente.

Calcul de l'angle : Le centre du cercle étant invariant malgré l'effet de halo, il a été utilisé pour calculer l'angle 'alpha' de la balle par rapport à l'axe central de la caméra, en exploitant le champ de vision horizontal (FOV).

Traduction en commandes : L'algorithme retourne désormais des valeurs exploitables en temps réel. Au lieu d'une simple information de présence, le code V8 génère des directives vectorielles prêtes à être injectées dans les moteurs du robot (ex : Balle Rouge détectée -> Tourner de 9° à droite et Avancer de 20cm).



II.4. Traitement de l'image

Intégration Matérielle et Perspectives

La Version 9 avait pour but de transposer ce code fonctionnel depuis l'environnement de développement (MacBook) vers le système embarqué du robot (Raspberry Pi avec module caméra Arducam IMX219). Le code a été mis à jour avec les caractéristiques optiques spécifiques de la nouvelle caméra (focale de 3.04mm, FOV de 62.2°) et adapté pour utiliser le backend vidéo Linux (cv2.CAP_V4L2).

Obstacle matériel :

Le projet est actuellement bloqué à cette étape en raison d'une défaillance matérielle. A l'exécution du programme `image_V9.py` sur la Raspberry, le terminal renvoie comme erreur qu'il ne détecte pas la caméra. Malgré de nombreuses sessions de dépannage au niveau de l'OS Linux (vérification des interfaces I2C, modification du fichier `config.txt` pour forcer la prise en charge des modules caméra legacy, tests avec `libcamera`), la caméra physique n'est pas détectée par la carte mère de la Raspberry Pi.

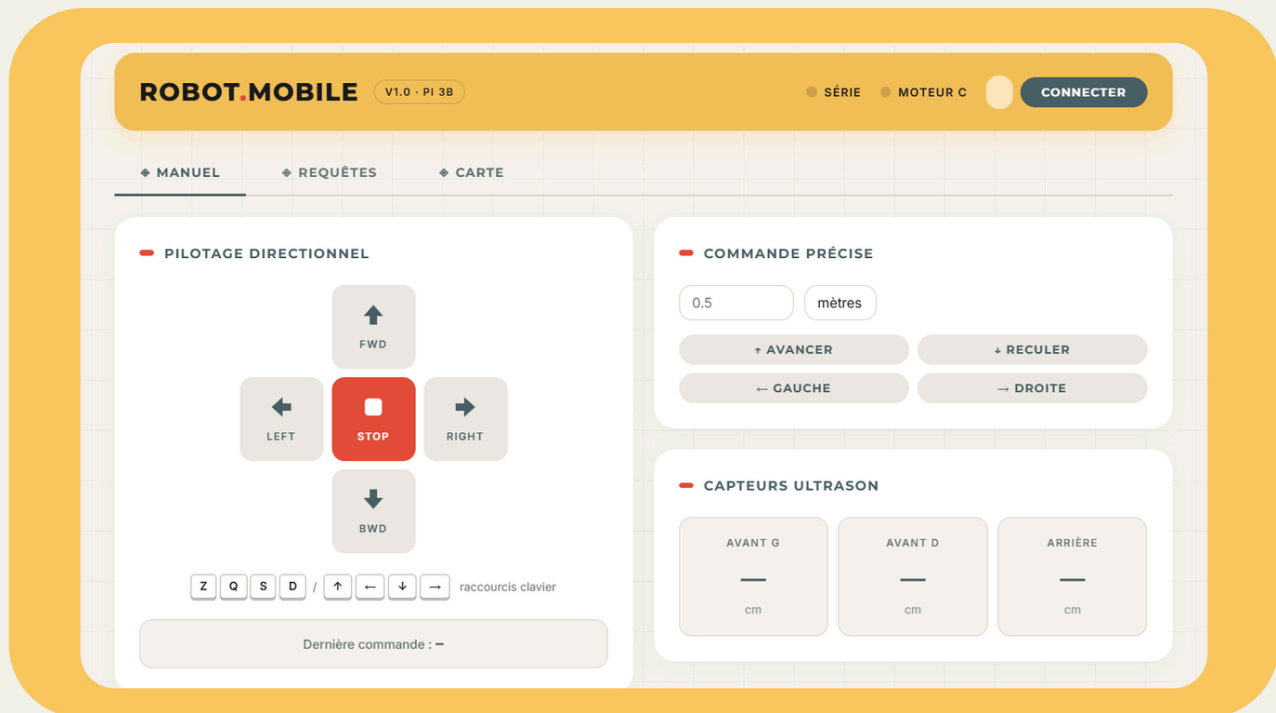
Conclusion et Marge de progression :

D'un point de vue logiciel et algorithmique, la mission est un succès total. L'algorithme V8 est robuste, optimisé pour le temps réel, et prêt à fournir la télémétrie nécessaire à la navigation autonome du robot.

La marge de progression actuelle n'est pas logicielle, mais logistique : il convient de remplacer la nappe de connexion CSI ou le module caméra défectueux de la Raspberry Pi. Dès que le matériel sera opérationnel, l'implémentation de la V9 se fera, permettant de clôturer l'intégration du sous-système visuel avec le châssis motorisé.

II.5. Interface Web

L'interface Web a pour objectif de représenter le centre de commande du robot pour l'utilisateur. Elle regroupe tous les modules développés lors du PFR2 et les rend exploitables. Le défi a été de rendre l'interface la plus intuitive et fonctionnelle possible et ainsi rendre l'utilisation du robot accessible pour tous.



Page principale de l'interface

L'interface a un bandeau principal avec un retour sur l'état de la connexion entre la Raspberry et l'Arduino et un bouton permettant de se connecter/se déconnecter avec le serveur. Trois onglets sont mis à disposition pour chaque fonctionnalité. Le premier onglet regroupe toutes les fonctionnalités de commandes manuelles. Le deuxième regroupe les requêtes textuelles et vocales avec un historique des commandes. Le troisième onglet permet de lancer le scan du LiDAR et d'afficher la carte générée. Cet onglet devait aussi servir à afficher les images prises par la caméra mais, par souci d'intégration, cette fonctionnalité n'a pas pu être implémentée.

L'interface a été séparée en trois onglets pour avoir une meilleure visibilité sur les fonctionnalités proposées par la plateforme. Aussi, l'utilisation du clavier pour piloter le robot peut s'effectuer depuis n'importe quel onglet, ce qui permet par exemple de piloter le robot en même temps que d'effectuer un scan de la pièce.

La description précise de chaque onglet est référencée dans le manuel d'utilisation.

II.5. Interface Web

Serveur Flask

Pour héberger l'interface et la rendre fonctionnelle avec la plateforme, nous avons utilisé un serveur Flask local sur la Raspberry Pi.

Le serveur Flask joue le rôle de passerelle centrale entre l'interface web, la carte Arduino et la Raspberry Pi. Il est responsable de l'exécution des fichiers de code sur la Raspberry Pi pour le traitement des requêtes, ainsi que du transfert des données de commandes vers la carte Arduino. Il permet également d'accéder à l'interface depuis n'importe quel appareil connecté au réseau local.

Le choix de Flask a été fait car il s'agit du framework le plus léger et le plus adapté au projet, notamment grâce à son implémentation en Python et sa simplicité d'intégration sur la Raspberry Pi, ne nécessitant qu'un seul fichier pour fonctionner.

III. Bilan individuel

Ny Aina RAHARITSIFA

Durant ce PFR2, j'ai principalement travaillé sur la partie pilotage du robot, en partant du firmware Arduino jusqu'à l'intégration complète dans l'interface web.

Ma première contribution a été le développement du code Arduino permettant de contrôler les moteurs. Il s'agissait de faire avancer, reculer et tourner le robot en réponse aux commandes reçues sur le port série, en traduisant chaque instruction en une durée de fonctionnement des moteurs.

J'ai ensuite travaillé avec un camarade sur la gestion des obstacles à l'aide des capteurs ultrason. Nous avons rapidement rencontré un premier problème : la vérification des distances ne s'effectuait pas en continu. Concrètement, si un capteur détectait un obstacle avant que le robot ne démarre, celui-ci refusait de bouger, ce qui était correct. En revanche, si l'obstacle apparaissait alors que le robot était déjà en mouvement, il ne s'arrêtait pas. Ce problème a été identifié et résolu assez rapidement en intégrant la lecture des capteurs dans la boucle principale du programme, via la machine à états, afin que la vérification soit permanente et non ponctuelle.

Une fois le comportement du robot stabilisé côté Arduino, j'ai simulé la communication entre la Raspberry Pi et l'Arduino en utilisant mon ordinateur personnel à la place de la Raspberry Pi, en le branchant directement sur l'Arduino via USB. Cela m'a permis de tester les modules de requêtes textuelles et vocales récupérés du PFR1, sans avoir à tout déployer immédiatement sur la Raspberry Pi, et donc de valider le pipeline complet, de la saisie de la phrase jusqu'à l'envoi des commandes à l'Arduino, dans un environnement plus accessible.

J'ai ensuite travaillé sur l'intégration de ces deux modes de pilotage dans l'interface web, afin que les requêtes textuelles et vocales puissent être envoyées directement depuis le navigateur, sans passer par un terminal.

Une fois l'ensemble des modules fonctionnels, nous avons tout déployé sur la Raspberry Pi pour effectuer les tests en conditions réelles, sans que le robot soit relié à un ordinateur. Cette phase nous a permis de procéder à la calibration des constantes de distance et d'angle : nous faisons rouler le robot sur une distance connue, mesurons l'écart avec la distance réellement parcourue, et ajustons les valeurs jusqu'à obtenir un comportement cohérent avec les consignes envoyées.

Ce projet m'a surtout appris que la théorie et la pratique sont deux choses bien différentes. En simulation, tout fonctionne comme prévu. Sur un robot physique, il suffit que la batterie soit moins chargée pour que tout soit à recalibrer. Cette expérience m'a donné une vision beaucoup plus réaliste de ce que représente le développement embarqué.

III. Bilan individuel

Antonin GERAIN

Durant le PFR2, j'ai contribué aux tests des composants, à la partie de la programmation Arduino, au développement de l'interface web et à l'intégration de toutes les fonctionnalités de la plateforme.

Au début du projet, une phase importante a été de tester tous les composants qui nous ont été fournis. L'objectif était de comprendre de façon globale leur fonctionnement respectif et leur utilisation au sein du robot. Aussi, cela nous a permis de faire un retour à notre cliente sur les composants défectueux et les remplacer.

Pour ce qui est de la première phase de développement, j'ai travaillé avec Ny Aina sur le programme Arduino. Nous avons décidé de travailler à deux sur cette partie car elle concerne l'objectif principal du robot : se déplacer en évitant les obstacles. Durant cette partie, je me suis plus concentré sur l'intégration des capteurs ultrasons et la gestion d'obstacle. J'ai aussi réorganisé le code pour séparer deux machines à état : une pour le pilotage via le Bluetooth et une pour le pilotage avec l'interface web. Cette séparation permet d'éviter tout conflit entre les types de communication. Travailler à deux nous a permis d'être plus efficace en nous répartissant les tâches et en partageant nos idées. Cela avait aussi du sens car Ny Aina avait travaillé durant le PFR1 sur les requêtes textuelles et moi sur l'interface, deux fonctionnalités liées étroitement pour l'intégration.

Pour la deuxième phase, je me suis concentré sur le développement de l'interface web. J'ai commencé par l'écriture du code HTML et CSS de l'IHM et réfléchi à la manière de la rendre esthétique (reprise de la même direction artistique que pour nos rapports et présentation) et intuitive pour l'utilisateur. Pour cette partie, je me suis reposé sur mes compétences personnelles acquises durant mes années en IUT et en faisant des recherches complémentaires sur ces langages. Une fois le code finalisé, je me suis penché sur la partie serveur Flask. Je ne connaissais pas ce type de serveur local, j'ai pu me documenter sur le sujet et développer le code en Python et ainsi comprendre son fonctionnement.

Pour la troisième phase, j'ai intégré l'interface web avec le serveur sur la Raspberry Pi puis y ai ajouté le reste des modules fonctionnels. L'objectif était de pouvoir exécuter tous les programmes et d'envoyer des consignes depuis l'interface et ainsi lier tous les modules entre eux et avoir une plateforme mobile totalement portable.

Ce projet dans sa globalité, m'a permis de renforcer mes compétences en programmation que ce soit en Python ou en C et à maintenir mes compétences en travail d'équipe. J'ai aussi appris à gérer des responsabilités et à rester flexible en cas de problème.

III. Bilan individuel

Adrien NOSEL

Durant ce PFR2, j'ai été responsable de l'intégralité du pôle traitement d'image et guidage visuel. Contrairement à mes collègues qui ont travaillé en binôme, j'ai dû gérer cette partie en autonomie, de la phase de recherche algorithmique jusqu'à la tentative d'intégration sur le système embarqué.

Au début du projet, ma mission était de transformer un flux vidéo brut en données de navigation exploitables. N'ayant pas de bases approfondies en vision par ordinateur, j'ai dû passer par une phase importante d'auto-formation sur la bibliothèque OpenCV. J'ai commencé par des tests sur images statiques pour comprendre les espaces colorimétriques. J'ai rapidement compris que le format RGB était trop sensible aux variations lumineuses de la salle, ce qui m'a poussé à basculer sur le format HSV. Ce choix a été déterminant pour la robustesse du code, car il m'a permis d'isoler la teinte de la balle indépendamment des ombres portées.

Pour la phase de développement, j'ai adopté une approche itérative en huit versions. Un point clé de mon travail a été la création d'un script de calibration personnalisé (`reglage_hsv.py`). Cet outil m'a permis de gagner un temps précieux en ajustant les seuils de détection en temps réel via des trackbars, plutôt que de modifier le code à chaque test. J'ai également dû résoudre des problèmes optiques complexes : la balle paraissait plus grande qu'elle ne l'était à cause d'un "halo" lumineux. J'ai résolu cela en intégrant des filtres d'érosion mathématique vu en cours d'image cette année et un gain de correction proportionnel, rendant la mesure de distance enfin cohérente avec la réalité.

Lors de la troisième phase, j'ai travaillé sur la géométrie du guidage. J'ai implémenté le calcul de l'angle de la balle par rapport au centre optique afin que le robot ne se contente pas de "voir" la cible, mais sache exactement de combien de degrés il doit pivoter pour l'atteindre. Cette partie m'a permis de lier mes compétences en mathématiques à la programmation Python.

Enfin, la phase d'intégration sur Raspberry Pi a été la plus formatrice, bien qu'elle ait abouti à une impasse matérielle. J'ai passé beaucoup de temps à configurer l'OS Linux, à manipuler les fichiers de configuration et les modules de la caméra (`libcamera`, `drivers legacy`). Bien que le matériel défectueux ait empêché la démonstration finale sur le robot, j'ai pu valider l'intégralité de ma logique sur ma station de travail, prouvant que le code est prêt pour un déploiement immédiat sur un matériel fonctionnel.

Ce projet m'a appris à gérer un projet technique complexe de A à Z en totale autonomie. J'ai appris que le plus grand défi en robotique n'est pas seulement d'écrire un code qui fonctionne mathématiquement, mais de savoir diagnostiquer si une panne provient de l'algorithme ou du matériel. Cette expérience a considérablement renforcé ma persévérance face aux problèmes techniques imprévus.

III. Bilan individuel

Lucas Haegel

Durant ce PFR2, j'ai été responsable des tests des composants, de la mise en place de l'environnement pour faire fonctionner la raspberry et les composants connectés dessus, de la mise en place du bluetooth et des déplacements du robot avec cette technologie et enfin de cartographier l'environnement avec le lidar. Pour réaliser ces tâches, j'ai travaillé avec l'ensemble des membres du groupe car mes parties ont nécessité beaucoup d'intégration et d'adaptation aux besoins qu'ils avaient.

Premièrement, pour le test des composants, j'ai utilisé les programmes de tests provenant des bibliothèques de Arduino pour valider ou non le fonctionnement de la carte MEGA 2560, du shield L293D (qui n'était pas fonctionnel), des moteurs et du module bluetooth HM10. Le shield avait un problème sur les bornes M2, M3 et M4 pour connecter les moteurs, ce qui a retardé la mise en place des tests de déplacements pour les moteurs le temps d'en obtenir un nouveau.

Ensuite le test de la raspberry ainsi que de la carte SD et de l'alimentation a été fait en même temps que la mise en place de l'environnement sur la raspberry. Le lidar et la caméra quant à eux ont d'abord été testés par Adrien Nosel avant que je reprenne leurs tests et intégration pour le décharger d'un surplus de travail car la mise en place du traitement de l'image était un bloc très dense.

Une fois les composants testés, j'ai rapidement installé une Debian sur la raspberry pour que les membres du groupe puissent travailler dessus. J'ai également configuré la raspberry pour que l'on puisse avoir un environnement graphique via VNCViewer. En revanche, nous nous sommes rendu compte trop tard que le modèle de raspberry fourni au départ ne correspondait pas aux besoins du projet car essentiellement car elle ne possédait qu'un port USB. Nous avons donc migré sur une raspberry pi 3B+ qui nous a fait prendre du retard au vu des configurations qu'il a fallu refaire mais qui nous a permis de continuer à avancer sur le projet.

Le bluetooth n'a pas été très compliqué à mettre en place, seulement j'ai perdu pas mal de temps sur la tentative d'implémentation du pilotage de la plateforme mobile en bluetooth par l'ordinateur. N'ayant pas réussi à obtenir un résultat convenable, nous avons finalement décidé de piloter les déplacements via la raspberry communiquant directement avec la carte MEGA 2560. Le faible temps de réaction pour piloter la plateforme nous a confortés dans cette solution.

Ensuite l'implémentation du lidar a été pour moi la partie la plus complexe de ce PFR2. Ayant récupéré cette partie pour décharger Adrien Nosel d'un gros module et car ce n'était pas pratique pour lui de travailler dessus avec ses appareils, j'ai dû lire beaucoup de documentation et me renseigner sur les technologies les plus adaptées pour réussir à cartographier efficacement l'environnement avec nos composants car je n'avais jamais travaillé sur ce type de technologie. La cartographie de l'environnement fonctionne bien, seulement la rotation du robot n'est pas gérée et crée une erreur de rotation sur l'image finale capturée de la map qui se superpose à l'ancienne capture.

III. Bilan individuel

Finalement l'implémentation de la caméra avec le bloc de gestion d'image sur la raspberry n'a pas abouti. Même si la caméra est reconnue sur la raspberry et que le module de traitement d'image fonctionne, nous n'avons pas réussi à joindre la capture d'image avec l'analyse. Il faudrait se renseigner davantage pour réussir l'intégration. Le reste du projet fonctionne bien et l'intégralité des autres modules ont réussi à fonctionner ensemble.

Pour aller plus loin, on pourrait une fois la caméra et l'analyse d'image implémentées, utiliser la caméra pour perfectionner la cartographie de l'environnement et éventuellement gérer l'évitement des obstacles avec le lidar une fois l'erreur de déplacement entre la position du robot réel et la position théorique calculée par SLAM réglée.

